

Para integrar **FloorplanVLM** dentro de tu arquitectura actual de **ArchiCheck** y resolver de raíz el problema de generalización con planos chilenos de distintas oficinas, debes cambiar el rol de tu código.

En lugar de intentar que OpenCV y los umbrales determinísticos adivinen dónde están los muros y las puertas en un plano con convenciones raras, **FloorplanVLM actúa como el motor principal de vectorización y traducción semántica**, transformando la imagen/PDF del plano directamente en un JSON estructurado de alta precisión que tu motor de reglas (OGUC) ya sabe leer.

La estrategia exacta de integración en tu pipeline de Python/Colab sigue esta estructura:

Paso 1: Aprovisionamiento y Despliegue (Self-Host)

Dado que FloorplanVLM es un modelo de código abierto basado en visión-lenguaje, necesitas correrlo en un entorno con GPU (Google Colab Pro con una A100/T4 o un servidor dedicado con PyTorch).

1. **Clonación del repositorio y descarga de pesos:** Accedes al repositorio oficial del paper (*FloorplanVLM* en GitHub/HuggingFace, liberado a partir de su publicación en 2026).
2. **Dependencias del entorno:** Configuras tu entorno con Python, PyTorch, Transformers (de Hugging Face) y las librerías de decodificación de secuencias espaciales que utiliza el modelo.

Paso 2: Ingesta del Plano y Generación del JSON Topológico

Tu pipeline actual pasa por rasterizar el PDF del plano arquitectónico a una imagen de alta resolución (PNG o JPEG). En lugar de aplicar `adaptiveThreshold` y filtros de color manuales, le pasas esa imagen directamente al VLM mediante un *prompt* estructurado o el tokenizador nativo del modelo.

El modelo procesa la imagen y emite como salida directa un **JSON estructurado** (similar al estándar que requiere tu motor de reglas). La estructura típica que devuelve y que aprovecha tu software incluye:

JSON



```
{
  "floorplan": {
    "units": "meters",
    "scale": 100,
    "walls": [
      { "id": "w1", "polygon": [[0.0, 0.0], [5.5, 0.0]], "thickness": 0.15, "type": "
      { "id": "w2", "polygon": [[5.5, 0.0], [5.5, 4.0]], "thickness": 0.10, "type": "
    ],
```

```

"openings": [
  { "id": "d1", "type": "door", "host_wall": "w2", "coordinates": [2.1, 4.0], "sw
  { "id": "win1", "type": "window", "host_wall": "w1", "coordinates": [1.2, 0.0],
],
"spaces": [
  { "id": "room_1", "name": "Baño", "polygon": [[0.0, 0.0], [2.0, 0.0], [2.0, 2.5
]
}
}

```

Paso 3: Conexión con tu Capa 3 (Motor de Reglas OGUC) y Capa 4 (UI del Arquitecto)

Aquí es donde FloorplanVLM encaja perfectamente con lo que ya construiste, optimizando tus dos últimas capas:

1. **Alimentación directa al Motor de Reglas:** Como FloorplanVLM agrupa de forma nativa los recintos (`spaces`) referenciando los muros y los vanos (`openings`) que los componen, tu motor de reglas en Python ya no tiene que "calcular" geometrías a ciegas.
 1. *Ejemplo:* Para validar la regla de ventilación OGUC (que exige que la superficie de la ventana sea un porcentaje mínimo del suelo del recinto), tu script simplemente toma `space.area` del recinto "Baño" y lo cruza con el `width` de la ventana vinculada (`win1`), sin preocuparse por si el plano usaba líneas rojas, azules o capas de AutoCAD distintas.
2. **Reutilización de la Interfaz de Validación Gráfica:** Tu interfaz actual (donde el arquitecto revisa y corrige la geometría antes de correr el análisis normativo) se mantiene exactamente igual, con una enorme ventaja: **ahora el arquitecto edita sobre un JSON limpio y estructurado**, en lugar de corregir falsos positivos de píxeles generados por OpenCV. Si el modelo detectó mal una puerta o el sentido de giro, la interfaz permite ajustar ese nodo del JSON de forma rápida.

Ventaja clave de esta arquitectura para ArchiCheck

Al delegar la extracción visual a un VLM entrenado en topología (FloorplanVLM) en lugar de depender de reglas heurísticas de color y ángulo:

- **Te olvidas de las 5 rondas de fallos por convenciones de oficinas:** El modelo entiende la *semántica* del plano (sabe que eso es una puerta sin importar si tiene un arco hecho con spline, una línea discontinua o un bloque dinámico complejo).
- **Mantienes tu propiedad intelectual intacta:** Tu valor diferencial sigue siendo tu **Motor de Reglas Normativas Chilenas (OGUC/PRC)** y la experiencia de usuario de validación, resolviendo el cuello de botella de la extracción geométrica con tecnología de punta (2026) sin reinventar la rueda de la visión por computadora.

